Programa:	BrainStorm 3.5
Protección:	TimeTrial 30 días
Objetivos:	Evitar que expire
URL descarga:	www.brainstormsw.com
Dificultad:	Sencilla
Herramientas:	OllyDbg 1.10 + PEiD 0.93
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinos] Fecha: 22-10-2005

Hola nuevamente a todos..!!

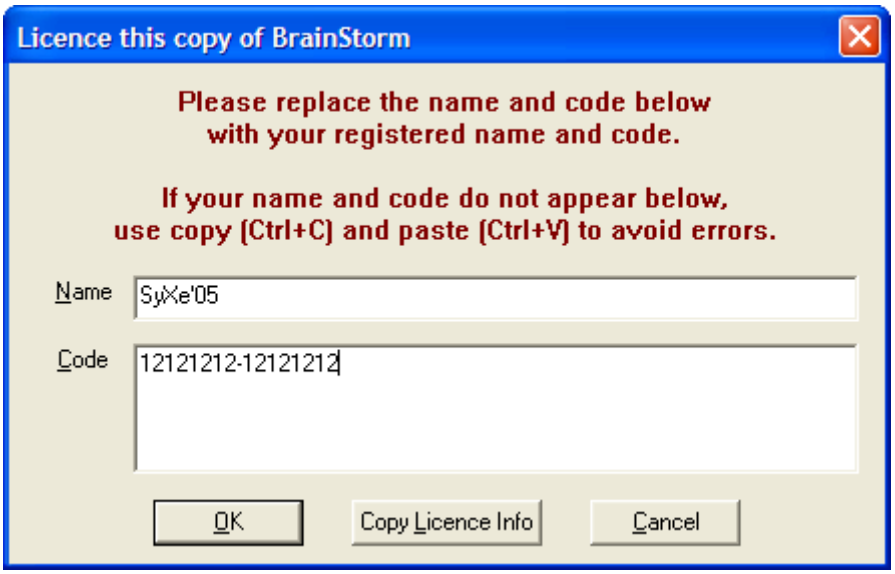
Hoy nos reunimos para abordar la protección de BrainStorm, en realidad todavía no sé para qué diablos se utiliza el programa porque no lo he utilizado ni nuna sola vez, eso sí, para crackear un programa no es necesario conocer el funcionamiento ni lo que hace, sino la protección en sí, en este caso se trata de una Cinderella a 30 días con la opción de introducir una combinación name/serial. Pasados los 30 días que nos dan de prueba ciertas opciones quedan deshabilitadas:

- Imprimir
- Salvar
- "Merge" que aún no sé lo que es ¿?
- Abrir archivos que no son del programa

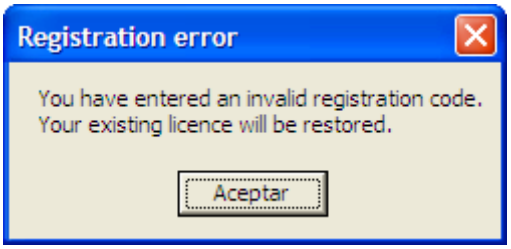
todo esto es lo que no se puede hacer cuando pasan los 30 días que hemos dicho antes, como vemos no nos deja hacer casi nada teniendo en cuenta las pocas opciones que ya se ve que tiene de por sí, pero bueno esto lo vamos a tratar con tanquilidad ahora mismo.

Cargamos el exe en Olly y damos RUN, de momento vamos bien, el programa carga perfecto así que desechamos la opción del antidebug, a ver a simple vista vemos ya diferentes puntos de ataque, podríamos intentar conseguir un serial válido o bien forzar la expiración del programa y a través de los mensajes que nos muestre evitar las limitaciones, pueden haber otros diferentes a estos está claro pero a mi son los que se me ocurren ahora mismo. He de decir que ya he traceado la rutina de comprobación del serial y obtener uno válido puede costar un buen rato, tiempo que yo no tengo así que lo que haré será dar una idea general de lo que hace y luego lanzar un ataque por las limitaciones expirando el programa.

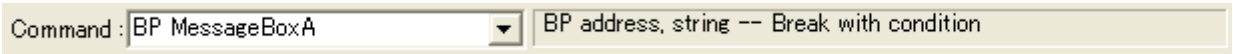
lo primero será ir al cuadro de registro y poner cualquier cosa que se nos ocurra acompañada de nuestro nombre de guerra ;-DDD



Aceptamos y nos sale el típico mensaje de error:



A partir de este mensaje podemos descubrir la rutina de verificación del serial fácilmente, pongamos un BP primero:



Y volvemos a repetir la jugada, introducimos los datos y al aceptar Olly para:

```
0012F2FC  00450C2D  CALL to MessageBoxA from Brain.00450C27
0012F300  00260526  hOwner = 00260526 ('Licence this copy of BrainStorm',class='#32770',parent=00040
0012F304  003F62E0  Text = "You have entered an invalid registration code.◙Your existing licence wil
0012F308  003F62A0  Title = "Registration error"
0012F30C  00000000  Style = MB_OK|MB_APPLMODAL
0012F310  003F6250
```

La llamada se realizó desde :450C27 como nos muestra la información del stack (pila) si vamos hasta allí veremos efectivamente la llamada a la api pero esto no lo tendremos muy en cuenta porque aquí seguramente se mostrarán más mensajes y no sólo este, entonces pulsamos Ctrl+F9 para llegar hasta el REt y lo superamos con F7/F8, caemos en:

```
0042C44C  .  50          PUSH EAX
0042C44D  .  51          PUSH ECX
0042C44E  .  8BCB        MOV ECX,EBX
0042C450  .  E8 AA470200 CALL Brain.00450BFF                mensaje de error..!!
0042C455  .  8B73 70     MOV ESI,DWORD PTR DS:[EBX+70]
0042C458  .  51          PUSH ECX
0042C459  .  8B4E F0     MOV ECX,DWORD PTR DS:[ESI-10]
0042C45C  .  8B11        MOV EDX,DWORD PTR DS:[ECX]
```

Si miramos por encima vemos estos saltos acompañados de un CALL delante:

```
0042C3AB  .  C68424 BC000( MOV BYTE PTR SS:[ESP+BC],0
0042C3B3  .  E8 C8A2FFFF   CALL Brain.00426680              debe retornar AL==1
0042C3B8  .  84C0          TEST AL,AL
0042C3BA  .v 74 12         JE SHORT Brain.0042C3CE          mal salto..!!
0042C3BC  .  8D4C24 20     LEA ECX,DWORD PTR SS:[ESP+20]
0042C3C0  .  E8 8B98FFFF   CALL Brain.00425C50
0042C3C5  .  83F8 05       CMP EAX,5
0042C3C8  .v 0F8D C001000( JGE Brain.0042C58E               buen salto..!!
0042C3CE  >  8B5C24 10     MOV EBX,DWORD PTR SS:[ESP+10]
0042C3D2  >  E8 B6310200   CALL Brain.0044F58D
0042C3D7  .  8B10          MOV EDX,DWORD PTR DS:[EAX]
```

Si se produce el primer salto en :42C3BA llegamos al mensaje de error pero si no se produce y salta además en :42C3C8 lo evitamos, entonces sabemos que:g

- en :42C3BA no debe saltar
- en :42C3C8 si debe saltar

Esto es equiparable a decir que el CALL de :42C3B3 debe retornar AL==1 y el de :42C3C0 retornar EAX<=5, de momento nos centramos en el primero de ellos, le ponemos un BP como en la imagen y volvemos a escribir los datos de registro ya que nos cierra el cuadro cada vez, aceptamos y rompemos en el BP, vamos traceando estudiando la rutina teniendo en mente que el valor de retorno debe ser AL==1, llegamos a esta parte:

```
0042680B  .  8BCD        MOV ECX,EBP
0042680D  .  8906        MOV DWORD PTR DS:[ESI],EAX
0042680F  .  E8 BCF8FFFF CALL Brain.004260D0               debe retornar AL==1
00426814  .  84C0        TEST AL,AL
00426816  .v 75 5B       JNZ SHORT Brain.00426873          AL==0 => no salta y deberia
00426818  .  8B7C24 14   MOV EDI,DWORD PTR SS:[ESP+14]
```

Nos encontramos con este subcall que debería retornar AL==1 y no lo hace, le ponemos un BP para entrar dentro la próxima vez que paremos, una vez dentro lo traceamos hsta ver donde recoge el valor que no nos gusta:

```
0042620B  .  56          PUSH ESI
0042620C  .  E8 CF8A0100 CALL Brain.0043ECE0               deja EAX==1
00426211  .  83C4 0C     ADD ESP,0C
00426214  .  85C0        TEST EAX,EAX
00426216  .v 0F85 AA00000( JNZ Brain.004262C6              no debe saltar..!!
0042621C  .  8D75 4F     LEA ESI,DWORD PTR SS:[EBP+4F]
```

Es en este Subcall que retorna EAX==1 cuando debería dejar EAX=0, repetimos la jugada otra vez poniendo un BP ahora en este call, entramos cuando pare y hacemos lo mismo que

antes, buscamos el lugar donde recoge los valores que no nos interesan y buscamos la manera de evitarlo:

```
0043ECE0  $  55          PUSH EBP
0043ECE1  .  8BEC        MOV EBP,ESP
0043ECE3  .  57          PUSH EDI
0043ECE4  .  56          PUSH ESI
0043ECE5  .  53          PUSH EBX
0043ECE6  .  8B4D 10     MOV ECX,DWORD PTR SS:[EBP+10]
0043ECE9  .v E3 27       JECXZ SHORT Brain.0043ED12
0043ECEB  .  8BD9        MOV EBX,ECX
0043ECED  .  8B7D 08     MOV EDI,DWORD PTR SS:[EBP+8]
0043ECF0  .  8BF7        MOV ESI,EDI
0043ECF2  .  33C0        XOR EAX,EAX
0043ECF4  .  F2:AE       REPNE SCAS BYTE PTR ES:[EDI]
0043ECF6  .  F7D9        NEG ECX
0043ECF8  .  03CB        ADD ECX,EBX
0043ECFA  .  8BFE        MOV EDI,ESI
0043ECFC  .  8B75 0C     MOV ESI,DWORD PTR SS:[EBP+C]
0043ECFF  .  F3:A6       REPE CMPS BYTE PTR ES:[EDI],BYTE PTR DS
0043ED01  .  8A46 FF     MOV AL,BYTE PTR DS:[ESI-1]
0043ED04  .  33C9        XOR ECX,ECX
0043ED06  .  3A47 FF     CMP AL,BYTE PTR DS:[EDI-1]
0043ED09  .v 77 05       JA SHORT Brain.0043ED10
0043ED0B  .v 74 05       JE SHORT Brain.0043ED12
0043ED0D  .  83E9 02     SUB ECX,2
0043ED10  >  F7D1        NOT ECX
0043ED12     8BC1        MOV EAX,ECX                    AQUI...!!!
0043ED14  .  5B          POP EBX
0043ED15  .  5E          POP ESI
0043ED16  .  5F          POP EDI
0043ED17  .  C9          LEAVE
0043ED18  .  C3          RETN
```

Este es el código del subcall, no es mucho y podríamos forzar a que retorne siempre el EAX==1 necesario parcheando en :43ED12, cambiando por ejemplo la instrucción:

```
0043ED12      8BC1          MOV EAX,ECX   POR:
0043ED12      33C0          XOR EAX,EAX
```

solventamos el valor de retorno del CALL:

```
0042C3B3   .  E8 C8A2FFFF   CALL Brain.00426680          ;debe retornar AL==1
0042C3B8   .  84C0          TEST AL,AL
0042C3BA   .  74 12         JE SHORT Brain.0042C3CE      ;mal salto..!!
```

Pero tenemos otro problema en este otro CALL:

```
0042C3C0   .  E8 8B98FFFF   CALL Brain.00425C50
0042C3C5   .  83F8 05       CMP EAX,5
0042C3C8   .  0F8D C0010000 JGE Brain.0042C58E                    ;  buen salto..!!
```

Ya que no retorna EAX<=5 sino EAX==FFFFCB7Dh, esto tendríamos que arreglarlo también para dejarlo todo listo, si entramos en el CALL vemos:

```
00425C50  $  56          PUSH ESI
00425C51  .  8BF1        MOV ESI,ECX
00425C53  .  85F6        TEST ESI,ESI
00425C55  .v 74 07       JE SHORT Brain.00425C5E
00425C57  .  8B46 70     MOV EAX,DWORD PTR DS:[ESI+70]
00425C5A  .  85C0        TEST EAX,EAX
00425C5C  .v 75 04       JNZ SHORT Brain.00425C62
00425C5E     33C0        XOR EAX,EAX
00425C60  .  5E          POP ESI
00425C61  .  C3          RETN
00425C62  >  6A 00       PUSH 0
00425C64  .  E8 E88C0100 CALL Brain.0043E951
00425C69  .  8B4E 70     MOV ECX,DWORD PTR DS:[ESI+70]
00425C6C  .  8B31        MOV ESI,DWORD PTR DS:[ECX]
00425C6E  .  83C4 04     ADD ESP,4
00425C71  .  2BF0        SUB ESI,EAX
00425C73  .  8B41 04     MOV EAX,DWORD PTR DS:[ECX+4]
00425C76  .  1BC2        SBB EAX,EDX
00425C78  .  6A 00       PUSH 0
00425C7A  .  81C6 7F51010( ADD ESI,1517F
00425C80  .  68 80510100 PUSH 15180
00425C85  .  83D0 00     ADC EAX,0
00425C88  .  50          PUSH EAX
00425C89  .  56          PUSH ESI
00425C8A  .  E8 418D0100 CALL Brain.0043E9D0
00425C8F  .  5E          POP ESI
00425C90  .  C3          RETN
```

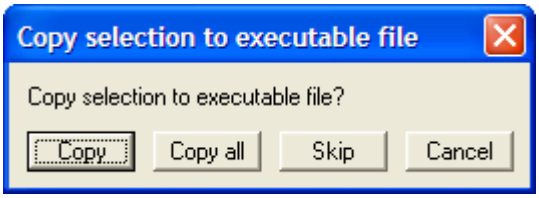
no es gran cosa, además no sería fácil obligarlo a retornar un valor superior a 5 en EAX, podríamos intentar cambiar la zona :425C5A - :425C5E de esta manera:

```
00425C50  $  56          PUSH ESI
00425C51  .  8BF1        MOV ESI,ECX
00425C53  .  85F6        TEST ESI,ESI
00425C55  .v 74 07       JE SHORT Brain.00425C5E
00425C57  .  8B46 70     MOV EAX,DWORD PTR DS:[ESI+70]
00425C5A     B8 05000000 MOV EAX,5
00425C5F     90          NOP
00425C60  .  5E          POP ESI
00425C61  .  C3          RETN
```

y nos aseguramos de que retorne EAX==5 siempre, ahora salvamos los cambios, recordad que tenemos además otra modificación:

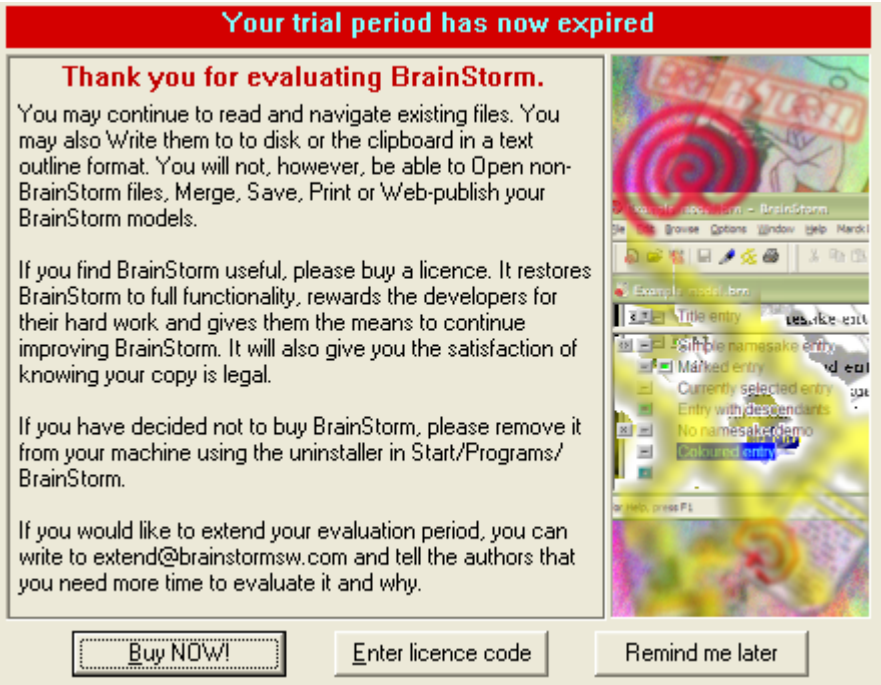
```
0043ED0D  .  83E9 02     SUB ECX,2
0043ED10  >  F7D1        NOT ECX
0043ED12     33C0        XOR EAX,EAX                    AQUI...!!!
0043ED14  .  5B          POP EBX
0043ED15  .  5E          POP ESI
0043ED16  .  5F          POP EDI
0043ED17  .  C9          LEAVE
0043ED18  .  C3          RETN
```

salvamos las dos con la opción "Copy to executable->All modifications" y aparecerá este pequeño cuadro donde debemos elegir "Copy all"

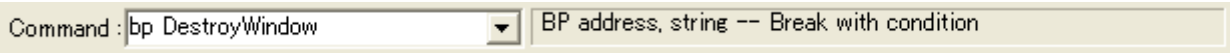


le damos un nombre tute.exe por ejemplo y probamos el archivo resultante y vemos que no va ;-(además las fuentes han quedado afectadas, el parche no es bueno. por suerte tenemos más recursos y a esto me refería antes cuando decía que había diferentes puntos de ataque, pondremos en práctica otro ;-)

Para realizarlo necesitamos que el prorgrama haya expirado, esto es fácil adelantamos el reloj del sistema cosa de un mes y ya está, si ahora lo intentamos ejecutar:



Esto es lo que nos aparece, eso sí nos deja entrar en el programa, lo que ocurre es que como ya sabemos ciertas opciones están deshabilitadas, si intentamos utilizar alguna de ellas el resultado es esta nag en pantalla, jejeje... y por su puesto no hace la función, lo bueno que tiene esto es que la propia nag nos va a llevar hasta el CALL o FLAG encargado del estado del programa (Trial, Registered, Expired, etc.,) lo que haremos será algún BP que nos ayude en la labor, vamos a "File->Merge file.." y nos aparece la nag, entonces ponemos nuestro BP preferido (con ella en pantalla eh!):



Cerramos la nagscreen y el Bp surge efecto:

```
77D1A185  B8 63110000   MOV EAX,1163
77D1A18A  BA 0003FE7F   MOV EDX,7FFE0300
77D1A18F  FFD2          CALL EDX
77D1A191  C2 0400       RETN 4
```

Estamos en User32.dll, vamos traceando hasta encontrar un punto interesante como este:

```
004041A2  .   FF15 1813480(  CALL DWORD PTR DS:[<&KERNEL32.GetTickCo  GetTickCount
004041A8  .   8B8E 4401000(  MOV ECX,DWORD PTR DS:[ESI+144]
004041AE  .   8986 4801000(  MOV DWORD PTR DS:[ESI+148],EAX
004041B4  .   8B11           MOV EDX,DWORD PTR DS:[ECX]
004041B6  .   FF92 4001000(  CALL DWORD PTR DS:[EDX+140]          call_nag_restriccion
004041BC  .   8B8E 4401000(  MOV ECX,DWORD PTR DS:[ESI+144]
004041C2  .   85C9           TEST ECX,ECX
004041C4  .   8B79 7C        MOV EDI,DWORD PTR DS:[ECX+7C]
004041C7  .v  74 07          JE SHORT Brain.004041D0
004041C9  .   8B01           MOV EAX,DWORD PTR DS:[ECX]
```

Aparecemos en :4041BC y venimos claro de :4041B6 que es donde aparece la nag, pero todavía no es lo que buscamos, si superamos el siguiente RET sí lo encontramos ;-)

```
0041BA65  .   E8 E6A10000    CALL Brain.00425C50
0041BA6A  .   85C0           TEST EAX,EAX
0041BA6C  .v  7F 12          JG SHORT Brain.0041BA80          saltar para MERGE => funka?
0041BA6E  .   E8 C7020500    CALL Brain.0046BD3A
0041BA73  .   8B48 04        MOV ECX,DWORD PTR DS:[EAX+4]
0041BA76  .   E8 C586FEFF    CALL Brain.00404140
0041BA7B  .v  E9 14060000    JMP Brain.0041C094
0041BA80  >   E8 083B0300    CALL Brain.0044F58D
0041BA85  .   8B10           MOV EDX,DWORD PTR DS:[EAX]
```

Ahí se ve que estamos en :41BA7B y venimos de :41BA76, ahora ¿qué hubiera ocurrido si hubiésemos saltado en :41BA6C? pues que nos hubiera puesto debajo del JMP y se hubiera ejecutado código que ahora no se va a ejecutar debido al salto que nos envía directamente a :41C094 muy lejos de donde estamos, lo que haremos será poner un BP en el CALL y forzarlo a retornar EAX<=5, si tenemos memoria este CALL ya lo conocemos de antes, estuvimos en él y lo habíamos parcheado para que retornara ya este mismo valor y no funcionó, quizá porque tambiñén hicimos más cambios, lo que está claro es que si retornara este valor no veríamos la nag al pulsar en la opción "Merge File" lo que haremos es trabajar con una copia del ejecutable original Brain.exe sin ningún cambio y comprobar el tema del call 425c50 de la imagen anterior.

```
0041BA65  .   E8 E6A10000    CALL Brain.00425C50
0041BA6A  .   85C0           TEST EAX,EAX
0041BA6C  .v  7F 12          JG SHORT Brain.0041BA80          saltar para MERGE => funka?
0041BA6E  .   E8 C7020500    CALL Brain.0046BD3A
0041BA73  .   8B48 04        MOV ECX,DWORD PTR DS:[EAX+4]
0041BA76  .   E8 C586FEFF    CALL Brain.00404140
0041BA7B  .v  E9 14060000    JMP Brain.0041C094
0041BA80  >   E8 083B0300    CALL Brain.0044F58D
0041BA85  .   8B10           MOV EDX,DWORD PTR DS:[EAX]
```

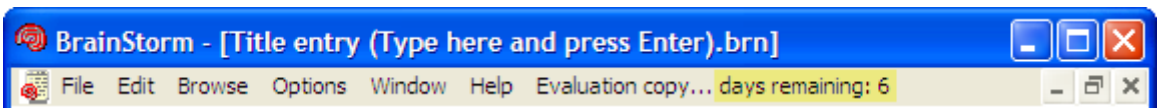
Le ponemos un BP y volvemos a elegir "Merge File" del menú, Olly salta y estamos en la dirección del call, o sea en :41BA65 entremos en él y veamos porqué no retorna EAX<=5 y de dónde toma el valor de reotorno:

```
00425C50  $   56             PUSH ESI
00425C51  .   8BF1           MOV ESI,ECX
00425C53  .   85F6           TEST ESI,ESI
00425C55  .v  74 07          JE SHORT Brain.00425C5E
00425C57  .   8B46 70        MOV EAX,DWORD PTR DS:[ESI+70]
00425C5A  .   85C0           TEST EAX,EAX
00425C5C  v   75 04          JNZ SHORT Brain.00425C62
00425C5E      33C0           XOR EAX,EAX
00425C60  .   5E             POP ESI
00425C61  .   C3             RETN
```

Es un recorte del CALL ya que es algo mayor (en otra imagen lo vimos completo) la segunda parte del CALL que es lo que hay detrás del RET y adonde apunta el salto no nos va a hacer falta para el parche por eso no la pongo, cambiamos esto sólo:

```
00425C50  $   56             PUSH ESI
00425C51  .   8BF1           MOV ESI,ECX
00425C53  .   85F6           TEST ESI,ESI
00425C55  .v  74 07          JE SHORT Brain.00425C5E
00425C57  .   8B46 70        MOV EAX,DWORD PTR DS:[ESI+70]
00425C5A      85C0           TEST EAX,EAX
00425C5C      33C0           XOR EAX,EAX
00425C5E      B0 06          MOV AL,6
00425C60  .   5E             POP ESI
00425C61  .   C3             RETN
```

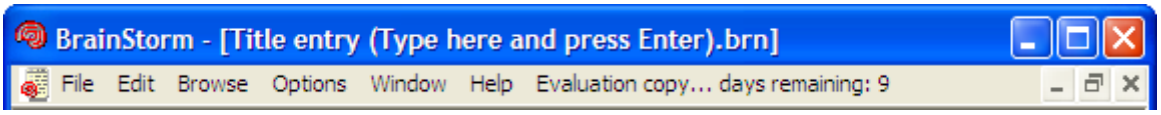
no aseguramos de salir con EAX==6 que cumple la restricción EAX<=5 perfectamente, salvamos los cambios y ejecutamos el nuevo exe recién modificado y...



Funciona perfectamente, volvemos a poder usar todas las funciones, lo que más me llama la atención es que nos vuelven a quedar exactamente 6 días y nosotros hemos parcheado el CALL para que retorne EAX==6 ¿no es curioso? parece que hemos localizado el contador de días, jeje.. haremos una segunda prueba para ver si fue casualidad cosa que dudo pero que bastantes, cambiamos el parche por este otro:

```
00425C50  $  56          PUSH ESI
00425C51  .  8BF1        MOV ESI,ECX
00425C53  .  85F6        TEST ESI,ESI
00425C55  .v 74 07       JE SHORT Brain.00425C5E
00425C57  .  8B46 70     MOV EAX,DWORD PTR DS:[ESI+70]
00425C5A     85C0        TEST EAX,EAX
00425C5C     33C0        XOR EAX,EAX
00425C5E     B0 09       MOV AL,9
00425C60  .  5E          POP ESI
00425C61  .  C3          RETN
```

Guardamos los cambios otra vez, ejecutamos el nuevo exe y...



Efectivamente hemos localizado el contador de usos del programa, como en Modo de Evaluación funcionan todas las opciones del programa ya lo tenemos crackeado porque al estar siempre en Modo Evaluación nunca expirará y podremos usar las funciones indefinidamente. Lo único que queda es eliminar el "Evaluation.. days remaining..." y todo eso de la barra de título además del Evaluation Copy del About, esto ocurre porque hemos congelado el Trial y no Registrado el programa, son los inconvenientes de no registrarlo ni sacar el serial válido.

El resto de detalles que ya hemos dicho "Evaluation" y cosas de estas se reparan todas jugando con el Resource Hacker.

Y dicho esto creo que los objetivos fueron cubiertos y podemos decir que hemos terminado el trabajo, si quedó alguna duda o pregunta me podéis escribir a:

syxe00@yahoo.es